C# / .NET Edition

Building AI Agents for PV Creation & Approval

Hands-on Workshop

AI Agent Bootcamp — Komatsu · Microsoft Agent Framework for .NET

Teerasej Jiraphatchandej, Microsoft MVP – AI,
www.facebook.com/teerasej | www.youtube.com/teerasej

The Business Problem

Payment Voucher requests are manual, error-prone, and slow to approve.

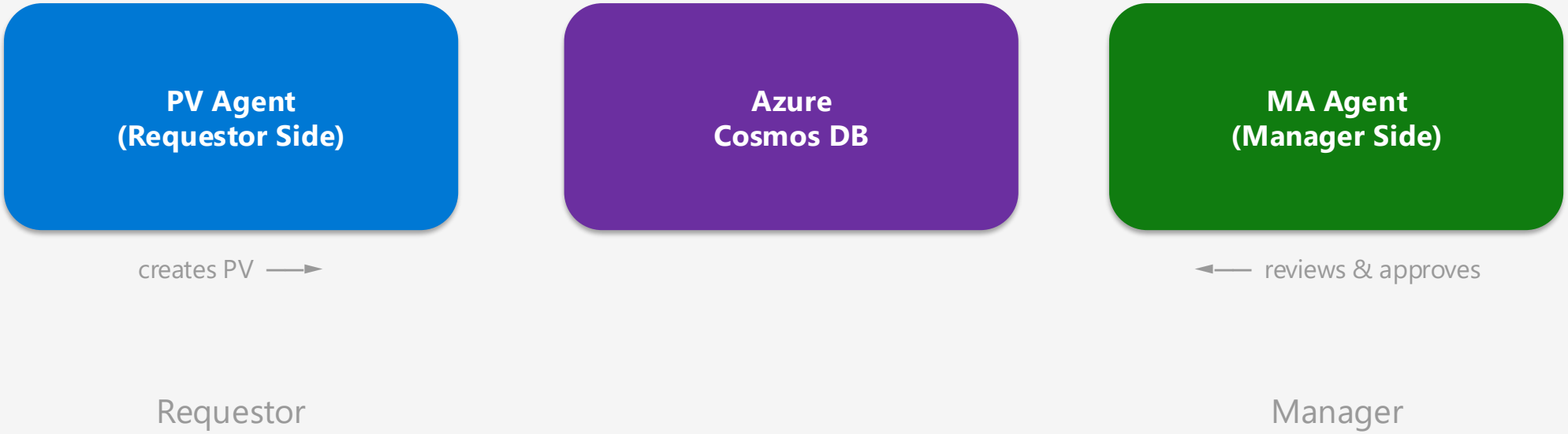
Manual data entry — requestors fill forms by hand, missing fields are common

No validation — incomplete or incorrect PVs reach the manager's desk

Slow approval chain — managers manually search through submissions

No single source of truth — data scattered across emails and spreadsheets

The Solution: Two AI Agents



We'll build two agents — one for creation, one for review. They share a database.

What is an AI Agent?

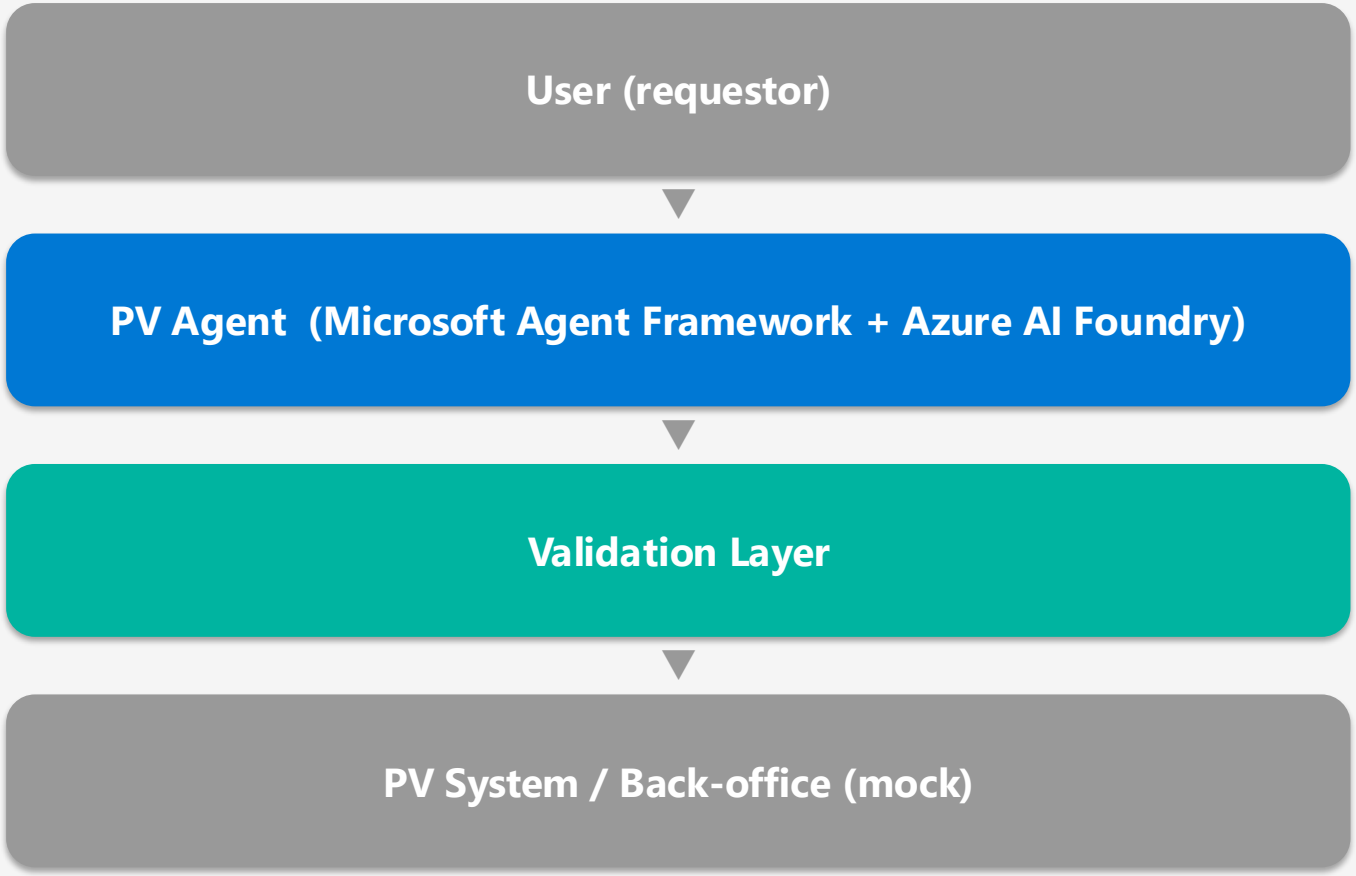
AI Agent = LLM + Instructions + Tools

LLM
(the brain)

Instructions
(the rules)

Tools
(the hands)

PV Agent Boundaries



The agent is a data assistant. It does NOT approve, does NOT access finance systems.

LET'S DO IT

D3-M1: Environment Setup (C#)

Create a Microsoft Foundry project
Deploy GPT-4.1 model
Configure appsettings.json
dotnet build → dotnet run (smoke test)

The System Prompt is Everything

It defines who the agent is, what it knows, and what it refuses to do.

Anatomy of a Good Instruction

1. Role	"You are PV Agent..."
2. Required Fields	List every field the agent must collect
3. Constraints	What the agent must NOT do
4. Output Format	The exact JSON template

Slot-Filling Conversation Pattern

The agent follows a slot-filling pattern — collect one field at a time, never invent data.

Instruction Design Rules

- ✓ Never ask for a field the user already provided
- ✓ Never invent data that the user did not provide
- ✓ State assumptions explicitly (e.g. "I'll use THB as the default currency")
- ✓ Only output the final JSON after user confirms the summary

Microsoft Agent Framework SDK — Minimal Pattern (C#)


```
// Create the AIAgent
AIAgent agent = new OpenAIClient(
    new ApiKeyCredential(apiKey),
    new OpenAIClientOptions { Endpoint = new Uri(endpoint) })
    .GetChatClient(deploymentName)
    .AsAIAgent(
        instructions: pvAgentInstructions,
        name: "PVAgent",
        tools: [...]);

// AgentSession holds the full conversation history
AgentSession session = await agent.CreateSessionAsync();

// Conversation loop — stream tokens
await foreach (AgentResponseUpdate update in
    agent.RunStreamingAsync(userInput, session))
{
    Console.Write(update.Text);
}
```

Three ingredients: OpenAIClient connection, instruction string, streaming conversation loop.

LET'S DO IT

D3-M2: Agent Role & Conversation Design (C#)

- Write the PV Agent instruction (system prompt) as a C# raw string literal
- Test 3 user stories in Foundry playground
- Implement the multi-turn loop with AIAgent + AgentSession

New Requirement: Budget Type

Finance says: every PV now needs a budgetType — either Expense or Investment.

Expense

Subscriptions, fees, training registrations, consumable items

Investment

Equipment purchases, tool licenses, infrastructure upgrades

Instruction Grounding vs. Code Logic

Code approach (C# if/else)

```
if (desc.Contains("monthly")) type = "Expense";
```

Misses edge cases

"one-time monitoring upgrade" → ???

Instruction approach

Add rule to the prompt (raw string literal)

LLM understands nuance

Handles ambiguity naturally

If the decision depends on understanding meaning, put it in the instruction — not code.

What Changes: Instruction Only

Zero code changes. Only the instruction string is updated.

REQUIRED FIELDS section of the raw string literal — add:

```
                - expense.budgetType: MUST be "Expense" or "Investment"
- "Expense" for recurring/operational costs
- "Investment" for one-time capital expenditures
```

The cheapest, fastest way to extend agent output — no recompile needed.

LET'S DO IT

D3-M3: Data Grounding via Instruction (C#)

Add budgetType classification rule to the C# raw string literal

Update OUTPUT FORMAT JSON template in the instruction

Test with 3 user stories (Expense, Investment, Ambiguous)

When Instructions Aren't Enough

Instructions handle rules.
But the agent can't look up facts from thin air.

Example:
User: "Project is IT Internal"
Agent needs: budget = 100,000 THB, remaining = 100,000 THB
→ The agent doesn't know this. It needs a data source.

Instruction vs. Tool — Decision Matrix

Classification rules, enums

→ Instruction grounding

Data lookup, calculations, I/O

→ Function tool (C# method)

[Description] Attribute Pattern (C#)

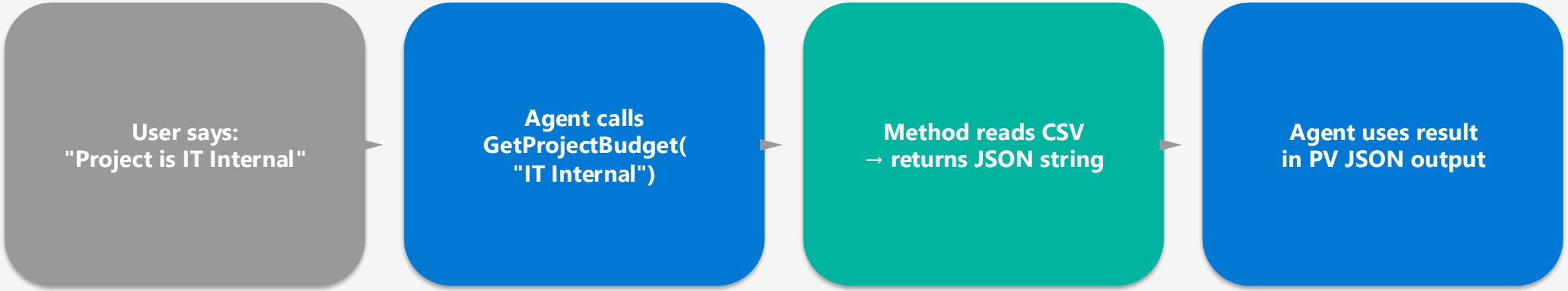

```
                    [Description("Look up the total budget and remaining budget
              for a given project from the CSV data file.")]
static string GetProjectBudget(
    [Description("The name of the project to look up")] string projectName)
{
    // Read CSV from AppContext.BaseDirectory/data/
    string[] lines = File.ReadAllLines(dataPath);
    // ... find row, return JSON string
    return $"{{"totalBudget": {budget}, "remainingBudget": {remaining}}}";
}

// Register with the agent:
tools: [ AIFunctionFactory.Create(GetProjectBudget) ]
```

① [Description] on the method — tells the agent what the tool does

② [Description] on each parameter — tells the agent what to pass

③ Return string — the agent reads the JSON result as conversation context

Data Flow: Agent Calls Tool



The agent autonomously decides when to call the tool. You don't trigger it manually.

LET'S DO IT

D3-M4: Function Tool for Budget Lookup (C#)

Implement GetProjectBudget with [Description] attributes

Register with AIFunctionFactory.Create(GetProjectBudget)

Test with known & unknown project names

Reversed Data Flow

Exercise 4 (Read)
Tool → returns data → Agent

Exercise 5 (Write)
Agent → passes JSON → SubmitPv()

Now the agent is the sender.
It assembles the complete PV payload and hands it to SubmitPv().

The Completeness Gate

Instruction rules that prevent premature submission:

1. Do NOT call SubmitPv until the user confirms AND all fields are filled
2. status must be "ReadyForSubmission" before calling the tool
3. Never submit with placeholders, empty strings, or zero amounts
4. Never submit a PV with status = "Draft"

The instruction acts as a gate — preventing premature submission.

Stub First, Wire Later

SubmitPv just writes JSON to the console. No database yet.

```
[Description("Submit PV data to system for approval")]
async Task<string> SubmitPv(
    [Description("Complete PV JSON as a string")] string pvJson,
    CancellationToken ct = default)
{
    Console.WriteLine("--- PV Submission Received ---");
    Console.WriteLine(pvJson);
    Console.WriteLine("--- End of PV Submission ---");
    return "PV submission received and ready for database insertion.";
}
```

Validate the payload shape before adding infrastructure complexity.

LET'S DO IT

D3-M5: Submit PV Tool (C#)

Implement SubmitPv with [Description] attributes

Register both tools: AIFunctionFactory.Create(GetAllProjectBudgets), AIFunctionFactory.Create(SubmitPv)

Add completeness gate rules to the instruction

Test full flow: collect → confirm → submit → console output

From Stub to Real Database

Before
SubmitPv → Console.WriteLine
(terminal only)

⟹

After
SubmitPv → CosmosClient
→ Cosmos DB

Same tool interface. Only the method body changes. The agent doesn't notice the swap.

Why Cosmos DB Serverless?

Pay-per-request — no minimum throughput charge

NoSQL — stores JSON natively (matches PV structure perfectly)

Shared container — both PV Agent and MA Agent use the same data

Partition key: /id — each document is its own partition (UUID via Guid.NewGuid())

Separation of Concerns

Agent Instruction	unchanged
Tool Interface	unchanged — SubmitPv(string pvJson) -> Task<string>
Tool Implementation	CHANGED — CosmosClient insert + stream API

Clean separation. The agent's behavior is stable even as the backend evolves.

LET'S DO IT

D3-M6: Store PV in Cosmos DB (C#)

Provision Cosmos DB Serverless account in the Azure Portal

Replace SubmitPv body with CosmosClient + CreateItemStreamAsync

Verify the document in Cosmos DB Data Explorer

PV Agent: What We Built

6 layers, incrementally added — one concept per exercise.

Two Sides of the Same Workflow

PV Agent
(completed ✓)

Azure
Cosmos DB

MA Agent
(building now)

Now we build the other side.

Same Patterns, Different Role

	PV Agent	MA Agent
Role	Collects & creates PV	Reviews & approves PV
Data direction	Writes to Cosmos DB	Reads from Cosmos DB
SDK pattern	Same (AIAgent + AgentSession)	Same (AIAgent + AgentSession)
Instruction	Different	Different

Agent specialization = same framework, different instructions.

The Manager's Instruction

Key differences from PV Agent:

Role: "Help managers review and explore PV requests" (not create)

Understands the PV JSON structure (reads the data contract)

Constraint: "Do NOT modify, do NOT invent data"

Constraint: "When manager asks to see PVs, ALWAYS call GetPvRequests tool"

Focus: Summarize, flag issues, answer questions about PV content

Reusing the SDK Pattern (C#)


```
            // Only the instruction and tools list differ from PV Agent
AIAgent agent = new OpenAIClient(
    new ApiKeyCredential(apiKey),
    new OpenAIClientOptions { Endpoint = new Uri(endpoint) })
    .GetChatClient(deploymentName)
    .AsAIAgent(
        instructions: maAgentInstructions,   // ← only this changes
        name: "MAAgent",
        tools: [...]);                        // ← added next

// Same session + streaming loop as PV Agent
```

Identical code structure. Only the instruction text changes.

LET'S DO IT

D4-M1: MA Agent Setup & Foundation (C#)

Write the MA Agent instruction raw string literal for the reviewer role

Implement conversation loop with AIAgent + AgentSession

Test with manager review scenarios (paste PV JSON, ask questions)

A Manager Needs Two Actions

Query

"Show me all pending PV requests"

→ GetPvRequests(status)

Act

"Approve PV pv-001"

→ UpdatePvApprovalStatus(id, status)

One agent, two tools — read and write.

Prototype with In-Memory Data (C#)

Before connecting to Cosmos DB, use a C# List<string> as the data source.

```
                    // Mutable list — each element is a raw JSON string (one PV document)
List<string> samplePvData =
[
    """{"id":"pv-001","pvTitle":"Monthly GitHub Copilot Subscription",
         "approval":{"status":"Pending"},...}""",
    """{"id":"pv-002","pvTitle":"Conference Registration Fee",
         "approval":{"status":"Approved"},...}""",
    """{"id":"pv-003","pvTitle":"Ergonomic Office Chairs",
         "approval":{"status":"Approved"},...}"""
];
```

Prototype fast. Validate tool patterns and agent behavior without database complexity.

Registering Multiple Tools (C#)


```
                                tools: [
    AIFunctionFactory.Create(GetPvRequests),
    AIFunctionFactory.Create(UpdatePvApprovalStatus)
]
```

The agent decides which tool to call based on the user's question.
"Show me pending PVs" → GetPvRequests
"Approve pv-001" → UpdatePvApprovalStatus

LET'S DO IT

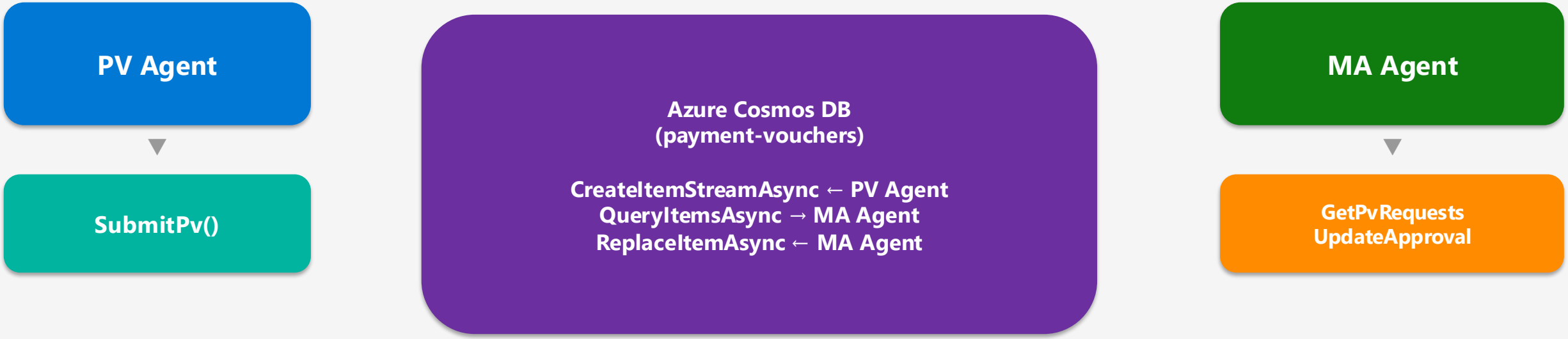
D4-M2: Function Tools for PV Review & Approval (C#)

Implement GetPvRequests and UpdatePvApprovalStatus with [Description] attributes

Register both tools with AIFunctionFactory.Create

Test: query pending → approve → verify approval

The Full Pipeline — End to End



Both agents now share live data. Submit a PV, then review and approve it.

What Changed: Implementation Only

Agent Instruction	unchanged
Tool Interface	unchanged — same [Description] signatures
Tool Implementation	CHANGED — List<string> → CosmosClient queries

Same pattern as PV Agent's Cosmos upgrade. The swap is always in the implementation layer.

LET'S DO IT

D4-M3: Connect MA Agent to Cosmos DB (C#)

- Add GetCosmosContainer() helper that returns the Container client
- Replace both tools with Cosmos DB QueryItemsAsync / PatchItemAsync
- End-to-end: PV Agent submit → MA Agent query → approve → verify

What We Built Today

PV Agent
6 exercises

Cosmos DB
shared container

MA Agent
3 exercises

1. System Prompt Design
2. Instruction Grounding
3. Function Tools (Read)
4. Function Tools (Write)
5. Cloud Persistence
6. Agent Specialization

Key Takeaways

1. The system prompt is your most powerful tool
2. Classification → instruction grounding. Facts → C# method with [Description].
3. Stub first (Console.WriteLine), wire the database later (CosmosClient)
4. Same AIAgent SDK + different instruction = different agent personality
5. Agents specialize — separate creation from review

From CLI to Web Chat UI

Take the completed PV Agent and expose it through a browser-based chat interface — with real-time streaming via Server-Sent Events (SSE).

You will learn: Switch SDK Console → Web · Per-session state with ConcurrentDictionary · Stream tokens via SSE

Architecture: One SDK Change, a New Surface

Lab 06 — Console App

Microsoft.NET.Sdk
(Console)

while(true) { Console.ReadLine() }

agent.RunStreamingAsync

Console.Write(update.Text)

⟹ upgrade

Lab 07 — Web App

Microsoft.NET.Sdk.Web
(only change in .csproj!)

POST /chat + Static Files

agent.RunStreamingAsync

SSE data: "token"\n\n

The agent logic (instructions + tools) is UNCHANGED. Only the transport layer switches from stdin/stdout to HTTP.

TODO 1 & 2 — Create the Agent & Session Store

Why .AsIChatClient() first?

ASP.NET Core adds Microsoft.Extensions.AI, introducing extra AsAIAgent() overloads. Inserting .AsIChatClient() resolves the type ambiguity explicitly.

TODO 1 — Create AIAgent:

```
AIAgent agent = new OpenAIClient(
    new ApiKeyCredential(apiKey),
    new OpenAIClientOptions { Endpoint = new Uri(endpoint) })
.GetChatClient(deploymentName)
.AsIChatClient()          // ← resolves overload ambiguity
.AsAIAgent(
    instructions: pvAgentInstructions,
    name: "PVAgent",
    tools: [
        AIFunctionFactory.Create(GetProjectBudget),
        AIFunctionFactory.Create(SubmitPv)
    ]);
```

TODO 2 — Session Store:

```
var sessions =
    new ConcurrentDictionary<string,
AgentSession>();
```

Why ConcurrentDictionary?

Thread-safe: multiple HTTP requests arrive simultaneously

Key = sessionId from browser (localStorage)

Value = AgentSession with full conversation history

New tab → new sessionId → fresh AgentSession

Page refresh → same sessionId → session survives

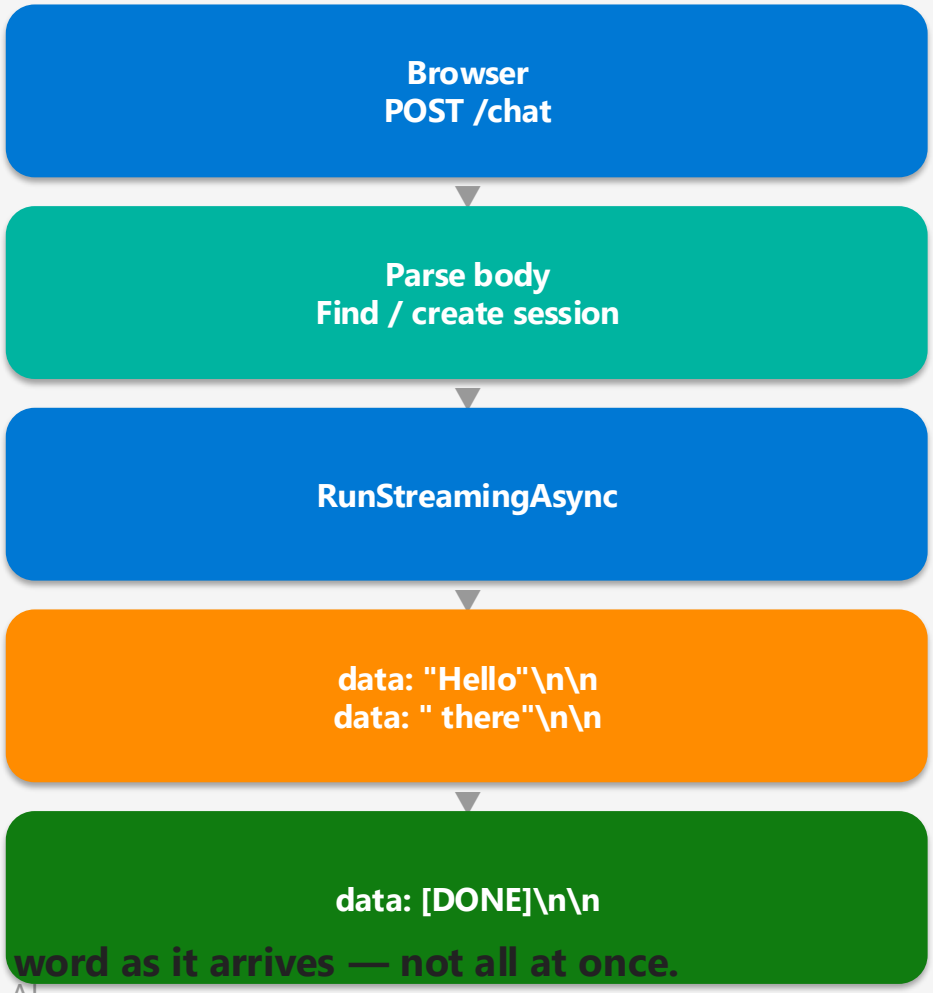
The CLI used a single AgentSession for one user. The web version isolates sessions so multiple tabs never share history.

TODO 3 — POST /chat: Stream Tokens as Server-Sent Events

Minimal endpoint — 5 steps:

```
using var doc = JsonDocument.Parse(bodyText);
var message   = doc.RootElement.GetProperty("message").GetString();
var sessionId = doc.RootElement.GetProperty("sessionId").GetString();

// b) Look up or create session
if (!sessions.TryGetValue(sessionId, out var session))
{
    session = await agent.CreateSessionAsync();
    sessions[sessionId] = session;
}

// c) SSE headers
ctx.Response.ContentType = "text/event-stream";
ctx.Response.Headers["Cache-Control"] = "no-cache";

// d) Stream tokens
await foreach (var update in
    agent.RunStreamingAsync(message, session))
{
    var data = JsonSerializer.Serialize(update.Text);
    await ctx.Response.WriteAsync($"data: {data}\n\n");
    await ctx.Response.Body.FlushAsync();
}
```

SSE token-by-token flow:



FlushAsync() after every token ensures the browser receives and renders each word as it arrives — not all at once.

LET'S DO IT

D3-M7: Integrate PV Agent into Web Chat UI

Copy appsettings.json.example → appsettings.json and fill credentials

Implement TODO 1: create AIAgent with .AsIChatClient().AsAIAgent(...)

Implement TODO 2: create ConcurrentDictionary session store

Implement TODO 3: map POST /chat with SSE streaming

dotnet run → open http://localhost:5000 → submit a full PV